Buscador de duplicados

Guía del código — cómo funciona por dentro

Recorrido por el código para quien quiera entenderlo o aprender a partir de él: la arquitectura por capas con puertos y adaptadores, los algoritmos (las 3 pasadas del escaneo, dHash, BK-tree, Union-Find), el patrón de hilos de la interfaz, los idiomas, el empaquetado y cómo se prueba todo.

Aplicación de escritorio para Windows · Python + CustomTkinter
Documento generado el 24/10/2026

Índice

1. Cómo leer este proyecto . . . 3

1.1. Mapa de módulos . . . 3

2. La lógica, explicada . . . 5

2.1. Detectar duplicados exactos: tres pasadas . . . 5

2.2. El hash: BLAKE2b y lectura en bloques . . . 5

2.3. El filtro: lista blanca y lista negra . . . 6

2.4. La decisión: qué copia se conserva . . . 7

2.5. Fotos parecidas: dHash + BK-tree + Union-Find . . . 7

2.6. Mover archivos con seguridad . . . 8

2.7. Historial y deshacer . . . 8

3. La arquitectura: puertos, adaptadores y raíz de composición . . . 9

4. La interfaz, explicada . . . 10

4.1. CustomTkinter y el arranque . . . 10

4.2. El patrón hilo + cola + after() (el más importante) . . . 10

4.3. Las tres pestañas . . . 10

4.4. Textos e idiomas . . . 10

4.5. Tema claro / oscuro . . . 11

4.6. Modo Sencilla / Avanzada . . . 11

4.7. La tabla (ttk.Treeview) . . . 11

4.8. Miniaturas e imágenes en Tkinter . . . 11

5. Empaquetado . . . 12

6. Cómo se prueba . . . 12

7. Añadir una función nueva: receta . . . 12

1. Cómo leer este proyecto

En esta guía te explico cómo está hecho el código, para que puedas entenderlo y aprender de él, igual que yo he ido aprendiendo. Está en el paquete `src/duplicate_finder/`, dividido en capas, y al lado hay una carpeta de pruebas:

Carpeta	Qué contiene	Regla
domain/	Reglas y datos: filtros, grupos de duplicados, selección, similitud.	No importa nada del resto. Sin entrada ni salida (ni disco, ni ventana).
application/	Los casos de uso (escanear, mover, reunir, historial...) y los puertos que necesitan.	Solo importa domain/ e i18n/. El disco, solo a través de puertos.
infrastructure/	Los adaptadores: disco, hashes, Pillow, JSON, registro.	Implementa los puertos. Nunca importa la interfaz.
presentation/	La ventana y todos sus widgets (CustomTkinter).	Solo pinta y reacciona a clics; delega en los casos de uso.
i18n/	Los textos en inglés y español.	No importa nada.
tests/	Pruebas con pytest, de varios niveles.	Crean archivos de verdad o usan dobles en memoria de los puertos.

Por dónde te recomiendo empezar a leer: `application/scan_duplicates.py` (el algoritmo central), luego `bootstrap.py` (cómo se conecta todo) y `presentation/main_window.py` (la carcasa de la ventana; cada pestaña es un componente en su `*_tab.py`). El resto son piezas que se entienden solas con su docstring.

1.1. Mapa de módulos

domain/	
duplicates.py	DuplicateFile, DuplicateGroup, ScanResult: el resultado de un escaneo.
filters.py	ScanFilter: qué incluir/excluir. CATEGORY_EXTENSIONS.
selection.py	DuplicateSelection: estado conservar/mover por grupo.
similarity.py	fotos parecidas: distancia de Hamming, BK-tree, Union-Find.
transfers.py	reglas de mover/copiar y qué hacer con lo que no son imágenes.
formatting.py, operations.py	tamaños legibles (KB/MB) y tipos de operación.
application/	
ports.py	las interfaces (Protocol): FileSystem, FileHasher, FingerprintReader, HistoryStore, SettingsStore.
scan_duplicates.py	ScanDuplicates: las 3 pasadas. Devuelve ScanResult.
find_similar_photos.py	FindSimilarPhotos: huellas + agrupar.
quarantine.py	MoveToQuarantine: validar, mover preservando estructura, CSV, historial.
gather.py, file_transfers.py	GatherImages y las primitivas mover/copiar por lotes.
history.py	OperationHistory: registro de operaciones + deshacer.
report.py, settings.py, start_selection.py	exportar CSV, ajustes, preselección.

application/	
services.py	Services: todos los casos de uso juntos, para la interfaz.
infrastructure/	
local_file_system.py	LocalFileSystem: os y shutil (con reintentos al borrar).
hashing.py	Blake2Hasher: hash BLAKE2b parcial y completo.
image_fingerprints.py	PillowFingerprintReader: dHash y firma de color.
history_store.py, settings.py	JsonHistoryStore y JsonSettingsStore (y la migración de los datos de la 1.0).
app_logging.py, data_folder.py	registro de errores y carpeta de datos.
presentation/	
main_window.py	MainWindow: cabecera, tema, idioma y modo. Crea las 3 pestañas.
exact_duplicates_tab.py	ExactDuplicatesTab: pestaña Duplicados exactos.
similar_photos_tab.py	SimilarPhotosTab: pestaña Fotos parecidas.
gather_images_tab.py	GatherImagesTab: pestaña Reunir imágenes.
scan_options_panel.py, preview_panel.py	los dos paneles de la primera pestaña.
background_job.py	BackgroundJob: hilo + cola + sondeo, para toda tarea larga.
results_table.py	ResultsTable: la tabla de duplicados exactos.
dialogs.py	las preguntas que comparten las tres pestañas al mover.
image_viewer.py, history_window.py, ...	visor con zoom, historial + Deshacer, diálogos.
thumbnails.py, widgets.py, styles.py, icons.py	miniaturas, piezas visuales, paleta clara/oscura y glifos.

2. La lógica, explicada

2.1. Detectar duplicados exactos: tres pasadas

Comparar el contenido de cada archivo con el de todos los demás sería lentísimo. En vez de eso filtro de lo barato a lo caro (`application/scan_duplicates.py`). Cada pasada es un método (`_size_pass`, `_partial_hash_pass` y `_full_hash_pass`) y `ScanDuplicates.__call__()` solo las encadena:

- **Pasada 1 — tamaños.** Recorro la carpeta con el puerto `FileSystem.walk` (por debajo, `os.walk`) y agrupo los archivos por su tamaño en bytes. Un tamaño único no puede tener duplicados, así que lo descarto sin abrir el archivo.
- **Pasada 2 — hash parcial.** Para los que comparten tamaño, calculo el hash BLAKE2b de solo los primeros 64 KB. Rápido, y descarta casi todos los que no son iguales.
- **Pasada 3 — hash completo.** Solo para los que además comparten hash parcial, hasheo el archivo entero. Los que coinciden aquí son idénticos byte a byte.

```
# clave = (tamaño, hash_parcial): dos archivos con el mismo hash
# parcial pero distinto tamaño nunca son el mismo archivo.
by_partial[(size, hp)].append(path)
...
finalists = {key: paths for key, paths in by_partial.items() if len(paths) > 1}
```

Nota. Optimización sutil: si un archivo pesa menos de 64 KB, su hash parcial ya cubre todo el contenido, así que me salto la Pasada 3 para él.

Podar carpetas al recorrer

El recorrido entrega tres cosas por carpeta: (ruta, subcarpetas, archivos). Si modificas la lista de subcarpetas *en el sitio*, no entra en las que quites — así se excluyen `node_modules`, `$RECYCLE.BIN`, etc. sin recorrerlas:

```
subdirs[:] = [d for d in subdirs if not filter.excludes_folder(d)]
```

2.2. El hash: BLAKE2b y lectura en bloques

Uso BLAKE2b (viene con Python, más rápido que SHA-256). Está en `infrastructure/hashing.py`, y el caso de uso lo recibe como puerto (`FileHasher`). El archivo completo lo leo de 64 KB en 64 KB para no cargarlo entero en memoria:

```
h = hashlib.blake2b()
with open(file_path, 'rb') as f:
    while True:
        block = f.read(65536)      # 64 KB
        if not block:              # b'' = fin del archivo
            break
        h.update(block)            # acumula
return h.hexdigest()
```

2.3. El filtro: lista blanca y lista negra

`ScanFilter` (en `domain/filters.py`) tiene dos mecanismos. Una **lista blanca** (`included_extensions`): si no está vacía, solo pasan esas extensiones ('buscar solo imágenes'). Y varias **listas negras** (extensiones, carpetas y nombres a ignorar, más un tamaño mínimo). El escaneo llama a `filter.excludes_name(...)` y `filter.excludes_folder(...)` y no cuenta ni lee lo que se excluye.

2.4. La decisión: qué copia se conserva

`DuplicateSelection` (en `domain/selection.py`) es un diccionario `(índice_de_grupo, ruta) -> 'keep' | 'move'`. Empieza todo en 'keep' (no se movería nada). La regla que impone la app: para poder mover, cada grupo debe tener **exactamente una** copia en 'keep'.

En modo Sencilla, `preselect_primary()` elige sola la copia principal de cada grupo con esta puntuación (gana la tupla más pequeña):

```
(looks_like_copy,  # 0 si el nombre no tiene 'copia', '(1)'...  1 si sí
 depth,            # nº de carpetas: cuanto menos anidada, mejor
 mtime,            # fecha: la más antigua suele ser el original
 len(path))        # a igualdad de todo, la ruta más corta
```

2.5. Fotos parecidas: dHash + BK-tree + Union-Find

Aquí no puedo comparar contenido (dos fotos parecidas tienen bytes distintos). Por cada imagen calculo una **huella** (`infrastructure/image_fingerprints.py`, con Pillow); la comparación de huellas es lógica pura (`domain/similarity.py`):

dHash de 64 bits

Reduzco la imagen a una miniatura de 9×8 en gris. Por cada fila, comparo cada píxel con el de su derecha: bit 1 si sube el brillo, 0 si baja. 8×8 = 64 bits. Mide la *forma*, no el color, por eso aguanta la recompresión y los filtros:

```
bits = 0
for row in range(8):
    for col in range(8):
        goes_up = gray[base+col] < gray[base+col+1]
        bits = (bits << 1) | (1 if goes_up else 0)
```

Distancia de Hamming

Cómo de distintas son dos huellas = cuántos bits difieren. `bin(a ^ b).count('1')`. 0 = iguales, 64 = opuestas. El deslizador de sensibilidad fija el umbral (~10).

BK-tree

Para no comparar todas las fotos contra todas, meto los dHash en un BK-tree: un árbol donde cada hijo va etiquetado por su distancia exacta al padre. Al buscar 'los que están a ≤ N', la desigualdad triangular permite **podar** ramas enteras: solo hay que mirar los hijos cuya etiqueta esté en `[d-N, d+N]`.

Union-Find

Si A se parece a B y B a C, deben acabar en el mismo grupo aunque A y C no se parezcan directamente. Union-Find (conjuntos disjuntos) une esas cadenas: cada elemento apunta a 'otro de su grupo' y siguiendo la cadena se llega al representante. `groups()` agrupa por representante.

2.6. Mover archivos con seguridad

Los casos de uso que mueven o copian (`quarantine.py`, `file_transfers.py`, `gather.py`) comparten las mismas reglas:

- **Nunca pisar.** Antes de mover, si el destino ya existe pruebo `nombre (1).ext`, `nombre (2).ext`... (`collision_free_path`).
- **Preservar estructura.** El destino de cada archivo es `carpeta_destino / relpath(origen, raíz)`, así se recrea la jerarquía de subcarpetas.
- **Mover, no copiar-y-borrar.** `shutil.move` (dentro del adaptador `LocalFileSystem`) hace un `rename` atómico si es el mismo disco; solo copia+borra si es otro disco (y ahí el original desaparece cuando la copia ya está a salvo).
- **Destino fuera del origen.** `validate_destination` y `validate_quarantine_folder` lo comprueban (si no, el próximo escaneo reencontraría lo movido).
- **Todo queda anotado.** El propio caso de uso escribe el CSV de registro y la entrada del historial; la interfaz no tiene que acordarse de hacerlo.

2.7. Historial y deshacer

Cada operación de mover/copiar se apunta (a través del puerto `HistoryStore`, en `%APPDATA%\BuscadorDuplicados\history.json`) con la lista de pares (origen, destino). `OperationHistory.undo(id)`:

- si fue **mover**: devuelve cada archivo al origen — pero si ya hay algo en el origen, lo avisa y no pisa.
- si fue **copiar**: borra las copias del destino. Es el único punto de todo el código donde se borra algo, y solo archivos que la app creó. El borrado reintenta un momento si Windows aún tiene el archivo en uso.

3. La arquitectura: puertos, adaptadores y raíz de composición

El truco que mantiene ordenado todo lo anterior es que los casos de uso no saben *cómo* se lee el disco, se calcula un hash o se guarda un JSON: piden esas cosas a través de **puertos**, que son interfaces (`typing.Protocol`) en `application/ports.py`:

```
class FileHasher(Protocol):
    partial_size: int
    def partial_hash(self, path: str) -> str: ...
    def full_hash(self, path: str) -> str: ...
```

Un caso de uso recibe sus puertos en el constructor y nunca crea nada por su cuenta:

```
class ScanDuplicates:
    def __init__(self, fs: FileSystem, hasher: FileHasher) -> None:
        self._fs = fs
        self._hasher = hasher
```

Los **adaptadores** de `infrastructure/` implementan esos puertos con el mundo real (`Blake2Hasher`, `LocalFileSystem`...). Y `bootstrap.py` es la **raíz de composición**: el único módulo que conoce todas las capas. Construye los adaptadores, los inyecta y le da a la ventana un objeto `Services` con todos los casos de uso:

```
def build_services() -> Services:
    fs = LocalFileSystem()
    history = OperationHistory(JsonHistoryStore(), fs)
    return Services(
        scan_duplicates=ScanDuplicates(fs, Blake2Hasher()),
        find_similar_photos=FindSimilarPhotos(fs, PillowFingerprintReader()),
        move_to_quarantine=MoveToQuarantine(fs, history),
        ...)
```

Qué se gana

- **Se prueba sin disco.** Las pruebas de los casos de uso les pasan dobles en memoria (`tests/fakes.py`): un sistema de archivos que es un diccionario, un hasher que usa el contenido. Son rápidas y deterministas.
- **La interfaz se podría cambiar** sin tocar la lógica, porque solo ve `Services`.
- **Cada cosa está en su sitio.** Si necesitas saber cómo se mueve un archivo, miras un adaptador; si necesitas saber cuándo, miras un caso de uso.

Nota. No todo tiene puerto: solo lo que toca el mundo exterior (disco, hashes, imágenes, historial, ajustes). Los cálculos puros (filtros, similitud, selección) son funciones normales en `domain/`; una interfaz ahí no aportaría nada.

Las reglas, comprobadas por una prueba

`tests/test_architecture.py` lee cada módulo con `ast` y falla si una capa importa algo que no debe (por ejemplo, `domain` importando `tkinter`, o `presentation` importando `infrastructure`), si el dominio o los casos de uso llaman a funciones que tocan el disco, o si algo fuera de `bootstrap.py` construye un adaptador. La propia prueba se comprueba con fragmentos de código que rompen cada regla.

4. La interfaz, explicada

4.1. CustomTkinter y el arranque

CustomTkinter es Tkinter con widgets repintados. `MainWindow` hereda de `ctk.CTk` y recibe el objeto `Services`. `app.py` (o `python -m duplicate_finder`) llama a `bootstrap.main()`: arranca el registro de errores, construye los servicios y abre la ventana.

4.2. El patrón hilo + cola + after() (el más importante)

Tkinter no es 'thread-safe': solo el hilo principal puede tocar widgets. Pero escanear tarda. La solución está en una sola clase, `presentation/background_job.py`, que usan todas las tareas largas (escanear, analizar, mover, reunir):

```
# hilo principal: lanzar. BackgroundJob programa el sondeo con after().
self._scan_job.start(
    lambda report: self._services.scan_duplicates(folder, progress=report, ...),
    what=f'Scan of {folder}')

# dentro de BackgroundJob, hilo secundario: NO toca widgets, solo la cola
result = work(lambda value: messages.put(('progress', value)))
messages.put(('done', result))

# hilo principal, cada 100 ms: vaciar la cola y avisar a la pestaña
kind, datum = self._messages.get_nowait()
if kind == 'progress': self._on_progress(datum)
elif kind == 'done':   self._on_done(datum)
```

Así la ventana no se congela, se ve el progreso y se puede cancelar (el hilo comprueba un `threading.Event` entre archivo y archivo). Si la tarea falla, `BackgroundJob` anota el error con su traza y avisa a la pestaña. Se prueba sin ventana (`tests/test_background_job.py`) sustituyendo `after` por un planificador que maneja la prueba.

4.3. Las tres pestañas

Un `ctk.CTkTabview` con tres pestañas. Cada una es un **componente**: una clase que hereda de `ctk.CTkFrame` y guarda su PROPIO estado (el resultado, las tareas en segundo plano...) en lugar de compartirlo con la ventana. La ventana solo las crea (`self.exact`, `self.similar` y `self.gather`) y les avisa con `apply_mode(advanced)` y `cancel_jobs()`. Cada pestaña repite el patrón: tarjeta de arriba con la carpeta, zona de progreso, panel de estadísticas, galería/tabla y barra inferior con el botón de acción. Las partes grandes son componentes aparte (`ScanOptionsPanel`, `PreviewPanel`), y las preguntas de mover (elegir la cuarentena, avisos) están en `dialogs.py`.

4.4. Textos e idiomas

Ningún texto de la interfaz está escrito en el código: cada uno se pide con `tr('clave', ...)`, y los textos viven en un catálogo por idioma (`i18n/locales/en.py` y `es.py`), un diccionario de claves a frases con marcadores:

```
"exact.moved_summary": "{moved} file(s) moved ({size})."      # en.py
"exact.moved_summary": "Se han movido {moved} archivo(s) ({size})."  # es.py

label.configure(text=tr('exact.moved_summary', moved=3, size='1.2 MB'))
```

El idioma se elige al arrancar (el guardado, o el de Windows, o inglés), antes de crear ningún widget; por eso el selector de la cabecera se aplica la próxima vez. `tests/test_i18n.py` comprueba que los dos catálogos tienen las mismas claves y marcadores, y que todo texto que pide el código existe. Los selectores (modo, tema, estrategia...) trabajan con claves estables y no con el texto visible, para que cambiar de idioma no rompa nada.

4.5. Tema claro / oscuro

En `presentation/styles.py` cada color es una tupla `(claro, oscuro)`. Los widgets de CustomTkinter aceptan esa tupla y cambian solos. Para lo que NO es CustomTkinter (la tabla `ttk.Treeview` y el `tk.Canvas` del visor) se usa `styles.col(tupla)`, que devuelve el color del tema activo, y se re-pinta a mano sobreescribiendo `_set_appearance_mode`:

```
def _set_appearance_mode(self, mode_string):
    super()._set_appearance_mode(mode_string)
    self._configure_ttk_style()    # ttk no cambia solo
    self._configure_tags()
```

Los botones deshabilitados pasan a gris (relleno y texto) con `styles.activate_button(btn, active)`, para que 'deshabilitado' no se confunda con un color de acción o de error. El tema elegido se guarda con el resto de los ajustes (`AppSettings`).

4.6. Modo Sencilla / Avanzada

`_apply_mode()` no crea ni destruye widgets: los quita y pone del grid con `grid_remove()` / `grid()`. Así el estado (lo que hubiera escrito el usuario) se conserva al alternar. La elección se guarda en los ajustes.

4.7. La tabla (ttk.Treeview)

Para los duplicados exactos se usa un `ttk.Treeview` (aguanta miles de filas). Al hacer clic hay que preguntarle a Tk qué hay debajo del ratón:

```
self.tree.identify_row(event.y)      # id de la fila
self.tree.identify_column(event.x)   # '#1' = columna 'Decisión'
self.tree.identify_region(x, y)      # 'heading', 'cell'...
```

Con eso: clic normal = seleccionar (y mostrar la vista previa); clic en 'Decisión' = alternar conservar/mover; doble clic = 'conservar solo esta'.

4.8. Miniaturas e imágenes en Tkinter

- Decodificar la imagen (Pillow) se hace en un hilo; produce un objeto PIL.
- Convertir a `ImageTk.PhotoImage` y pegarlo en la fila se hace en el hilo principal.
- **Hay que guardar una referencia** a cada PhotoImage (`self._img_refs[iid] = photo`). Si no, Python la recolecta y la imagen desaparece de la pantalla.

5. Empaquetado

`build.spec` configura PyInstaller: un solo archivo, sin consola, con los datos de CustomTkinter incluidos y `icon.ico` como icono (el código está en `src/`, que se añade a la ruta de búsqueda). `build_exe.py` lo ejecuta y comprueba el resultado.

El icono es una lupa **dibujada por código** (`presentation/icons.py:window_icon`) sobre un `tk.PhotoImage`, y `gen_icon.py` arma el `.ico` multi-tamaño a mano con `struct`. Nada de imágenes externas.

6. Cómo se prueba

Hay varios niveles. Las pruebas de comportamiento sobre archivos no usan *mocks*: cada una crea archivos reales en una carpeta temporal (`tempfile.TemporaryDirectory`), ejecuta la función de verdad y comprueba el estado en disco. Las de los casos de uso (`tests/test_use_cases.py`) usan los dobles en memoria de `tests/fakes.py`. Las de interfaz montan la ventana entera, simulan los diálogos (`filedialog` y `messagebox` se sustituyen por funciones que devuelven un valor fijo) y pulsan los métodos de los botones. Además hay pruebas de la arquitectura, de los catálogos de idioma y de la migración de los datos de la versión 1.0.

```
pytest                    # todo
pytest -m "not gui"       # solo lógica, sin abrir ventanas
pytest --cov              # con cobertura (el CI exige 90 % lógica, 70 % interfaz)
python tests/check_perceptual.py "D:\Fotos" 10   # con datos reales
```

7. Añadir una función nueva: receta

- **1.** Escribe la regla pura en `domain/` y el caso de uso en `application/` (una clase, con los puertos que necesite en el constructor). Si necesita algo del mundo exterior que no existe aún, añade un puerto y su adaptador en `infrastructure/`.
- **2.** Conéctalo en `bootstrap.py` y añádelo a `Services`.
- **3.** Añade pruebas: del caso de uso con dobles en memoria y, si toca archivos de verdad, con carpetas temporales.
- **4.** En `presentation/`, añade los widgets y un handler que llame al caso de uso. Si la tarea es larga, lánzala con `BackgroundJob`. Cada texto nuevo va a los dos catálogos con `tr()`.
- **5.** Si mueve o copia archivos, hazlo pasar por `FileTransfers` y regístralo en el historial para que se pueda deshacer.
- **6.** Ejecuta `pytest`, `ruff check .` y `python -m mypy`. Todo en verde antes de dar por hecho nada: incluida la prueba de arquitectura, que te avisa si has puesto algo en la capa equivocada.